examace the interacting scheduler

current model - constraints, Qiskit comparison & actual source

read pages 1-12 for the presentation route. the code appendix contains the current scheduler, benchmark runner & app. earlier guides describe the preserved allocation experiments.



one saved profile - both solvers receive the same scores, weights & budget

the problem now

students choose among study opportunities that compete for time. a practice session can depend on earlier review. a session can overlap another option or occur after an exam. the planner must select a compatible combination, rather than giving every next block to a separate topic.

what you can demonstrate

answer the short diagnostic, choose availability & exam days, then build an exam-week schedule. show preparation before practice, time for breaks & excluded session reasons. the Practice tab links exercises to selected topics. Quantum lab compares the same saved scheduling problem with exact, greedy & QAOA results.

the model uses up to 18 session choices. this gives 262144 possible on/off combinations. it is a bounded hackathon case study with measurable interactions between choices.

02 - one bit per study opportunity

a bit is a value restricted to 0 or 1. each candidate session gets one bit. bit 1 means select the session. bit 0 means leave it out. a register is a group of bits. eighteen bits represent all subsets of eighteen sessions.

candidate	binary decision
review integrals at 09:00	x0
practice series at 09:30	x1
practice integrals at 11:00	x2

if the first two overlap, both cannot be selected. if x2 needs the integrals review, selecting x2 requires x0 as well. the decision about one bit now affects what other bits are allowed to do.

```
index = 5

# binary 101 selects sessions 0 & 2

bit_i = (index >> i) & 1
```

>> shifts an integer right. & 1 reads its final bit. numpy applies this operation across all basis indices to build the small simulator cost table. qiskit displays the highest-index bit on the left, so int(bitstring,2) correctly restores our integer index.

03 - the actual constraints

availability

a selected session must fit entirely inside one available window.

deadlines

its end must be before the selected exam deadline for that topic.

overlap & breaks

selected sessions must leave at least 15 minutes between them. the budget includes 15-minute breaks between sessions on the same day.

preparation

a practice session with preparation requirements needs at least one listed review session selected earlier.

total budget

sum focused minutes & required break minutes, then compare with the total available budget.

the app exposes the planning days, daily availability, exam days, preparation switch & focus length. focus lengths are 15, 30 or 45 minutes. changing any setting clears the previous schedule & comparison.

the model uses alternative earlier reviews for a preparation requirement. one suitable earlier review satisfies that requirement. this keeps the schedule flexible across days.

04 - how sessions earn a score

```
gap = topic_weight*(1-initial_topic_score)

days = max(0,(exam_time-session_start)/1440)

urgency = 1 + 1/(1+days)

value = gap*urgency*activity_factor*(minutes/45)
```

a lower initial topic score gives a larger gap. urgency increases as the exam approaches. the activity factor is 1 for review & 1.4 for practice. duration scales the opportunity value. these are explicit planning choices that can be inspected in code.

repeat value

when two selected sessions repeat the same topic & activity, subtract .2 times the smaller session value. this encourages useful variety while still allowing repeated work. these numerical values define a priority heuristic; empirical learning evaluation remains future work.

why greedy can miss the best schedule

greedy selects the next compatible improvement. a high-scoring session can block two sessions whose combined score is higher. a preparation session can also unlock a valuable practice opportunity. considering the whole combination matters.

in the recorded mixed-gap 18-choice case, exact scored 12.815 & greedy scored 10.936. both schedules obeyed the constraints. that difference demonstrates the effect of interacting choices.

05 - energy & the classical references

```
energy = -utility + P*violations

P = 1 + sum(all_positive_session_values)
```

a coefficient is a multiplier, such as P. negative utility means lower energy corresponds to higher schedule value. violations count availability, deadline, overlap, preparation & budget failures. budget excess is rounded upward into 15-minute units before its penalty is squared.

P exceeds all possible positive session value. even one violation therefore costs more than any score can compensate. the empty selection is feasible with zero score. this makes a lowest-energy answer feasible under the supplied rules.

exact

the small exact reference evaluates every candidate subset, finds the highest feasible score & resolves ties with less used time followed by earlier session choices. the array implementation is fast at eighteen choices. its memory cost grows exponentially.

greedy & uniform

greedy adds the next compatible score improvement until none remain. uniform sampling measures a simple randomly sampled comparison using the same number of final samples. those references help show which properties come from the model & which come from the optimization process.

06 - qubits & amplitudes

a qubit is measured as 0 or 1 in the basis used here. before measurement it has amplitudes. for one qubit write a*|0> + b*|1>. probability of outcome 0 is abs(a)**2; probability of outcome 1 is abs(b)**2. all probabilities sum to one.

an amplitude can be negative or complex. Python writes the imaginary unit as 1j. it satisfies (1j)*(1j)=-1. for amplitude u+v*j, the squared magnitude is u*u+v*v.

the simulator stores one amplitude for each basis bitstring. eighteen qubits have 262144 amplitudes. one shot yields one bitstring. Statevector gives us the complete amplitudes because this project uses an ideal classical simulator.

phase & interference

phase is the direction or sign of an amplitude. +1 & -1 have equal magnitude but different phases. a later gate combines amplitudes so contributions can reinforce or cancel. this is interference.

```
hadamard output a = (a+b)/sqrt(2)

hadamard output b = (a-b)/sqrt(2)
```

start at a=1,b=0. one hadamard gives equal amplitudes. applying it again cancels the outcome-1 amplitude & returns outcome 0 with certainty. changing the relative sign before that second gate changes the cancellation. this is why phase information can affect later measurement probabilities.

07 - the actual Qiskit operations

```
preparation = QuantumCircuit(n)
preparation.ry(2*arcsin(sqrt(occupancy)), range(n))
initial_state = Statevector.from_instruction(preparation)
```

occupancy is an initial probability of selecting a session. the angle creates amplitudes sqrt(1-occupancy) for 0 & sqrt(occupancy) for 1. a small time budget starts with fewer sessions likely to be selected. this uses the budget & average duration, without using the exact answer.

```
phased = Statevector(initial.data*exp(-1j*gamma*E/P))
```

the cost operation multiplies every amplitude by a unit-magnitude phase. energy changes relative phase while preserving immediate basis probabilities. dividing by P sets the energy scale for the angle. numpy evaluates this diagonal operation directly for the simulator.

```
mixer = QuantumCircuit(n)
mixer.rx(2*beta, range(n))
state = phased.evolve(mixer)
```

the mixer uses Qiskit X rotations. Rx(theta) contains a half-angle in its definition, so 2*beta implements the standard qaoa mixer strength. evolving the phase-adjusted state lets interference change measurement probabilities. this is one cost/mixer layer, p=1.

08 - optimization & sampling

```
expected_energy = dot(state.probabilities(), energies)
```

expected energy is a probability-weighted average over candidate energies. it differs from the best observed candidate. scipy changes beta & gamma to reduce the average. we use one starting angle vector, at most 30 evaluations & a fixed seed for each recorded run.

```
state.seed(seed)

counts = state.sample_counts(1024)
```

Qiskit draws the final measurements. each key is a sampled session-selection string & each count is its frequency. the decoder rejects invalid selections & chooses the highest-scoring feasible sample. a failed sample set is saved explicitly.

the refinement we actually made

a separate short-budget run initially returned no feasible sample. we changed the initial state to reflect the budget. we retained the first benchmark & reran the same case/seed matrix. this lets us compare an actual iteration rather than reporting only a favorable run.

the cost table is constructed classically. Qiskit performs state preparation, mixer evolution & sampling. every reported quantum result is local ideal simulation. hardware implementation, deeper circuits & broader evaluation are future work.

09 - the measured comparison

measurement	observed result
fixed runs	27 - three profiles, three sizes, three seeds
sizes	12, 15 & 18 candidate sessions
feasible sampled schedule	27/27 runs
mean sampled/exact score	85.1%
exact score matched	4/27 runs
mean feasible-shot rate	8.2%
earlier starting-state result	2.8%
mixed-gap exact / greedy	12.815 / 10.936

case budgets vary with size, so this matrix is a fixed case study rather than a controlled qubit-scaling experiment. all runs remain in the recorded data, including weaker samples. the benchmark supports statements about these cases. it does not establish general reliability, hardware speed or improved learning.

the practical product is a constraint-aware planner. the quantum contribution is the binary model, Qiskit simulation, starting-state refinement & reproducible comparison. exact remains the reference score ceiling.

10 - presenting & tracing the code

"examace builds study schedules around availability, preparation & exams. those choices interact: a high-value session can block a better combination. our exact reference outscored greedy in a recorded case. we encode each candidate session as a qubit & compare a bounded Qiskit QAOA simulation on the same saved inputs. across 27 runs, all found a feasible sampled schedule. average sampled score was 85.1% of exact. changing the starting state increased mean feasible-shot rate from 2.8% to 8.2% in the fixed matrix. the app connects this experiment to a usable schedule & reviewed practice."

file map

scheduler.py owns models, candidate generation, constraints, scoring & solvers. app.py owns settings, state, tabs & downloads. dashboard.py renders schedules & practice. benchmark_scheduler.py constructs fictional scored cases & runs all configured seeds. the saved report contains the original inputs & actual samples.

primary sources

qaoa - https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm

statevector - https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector

optimizer - https://docs.scipy.org/doc/scipy/reference/optimize.minimize-cobyla.html

the appendix contains current source with explanations before important expressions. validation was run locally; new checker files are kept outside the submitted repository.

11 - trace one tiny problem

suppose A is a review worth 2, B is its later practice worth 4, C is an unrelated review worth 3. A & C overlap. B requires A. each lasts 45 minutes. the budget is 105 minutes, including a 15-minute break between A & B.

mask / selected	utility	feasible?
000 / none	0	yes
001 / A	2	yes
010 / B	4	no - missing A
011 / A & B	6	yes - 45+15+45=105
100 / C	3	yes
101 / A & C	5	no - overlap
110 / B & C	7	no - missing A
111 / all	9	no - overlap & budget

bit 0 belongs to A. its displayed position is on the right. 011 is decimal 3: 1*1 + 1*2 + 0*4. the exact solver selects A & B. greedy first selects C because B is unavailable without preparation & C scores above A. C then blocks A. this illustrates why local improvements can miss the best combination.

set P=1+2+4+3=10. A & B have energy -6. B alone has energy -4+10=6. every invalid choice pays at least 10, so the energy minimum stays feasible. the actual program also reduces repeated topic/activity value & squares rounded budget excess.

the phase step assigns each mask a phase using its energy. the mixer moves amplitude between masks that differ by one bit. scipy changes two angles to lower expected energy. the final shots may include invalid masks; the decoder filters them. the selected sampled schedule must have appeared in those shots.

12 - read the arrays without guessing

landscape is a lookup table for a small simulation. with n=3 sessions it has 8 rows. indices holds [0,1,2,3,4,5,6,7]. bits has 8 rows & 3 columns. row 3 is [1,1,0], so it selects sessions 0 & 1. values has one number per session. bits @ values multiplies each row by those values & sums the selected scores.

```
bits[:, i] # column i, every candidate subset

bits[index, i] # one selected/not-selected decision

data["valid"][index] # whether this subset passes all constraints
```

numpy broadcasting lets one expression evaluate many subsets at once. indices[:,None] turns a flat index list into a column. shifting it by np.arange(n) produces every bit position as a separate column. int8 stores each extracted 0/1 compactly. uint32 stores the subset numbers. these storage types are adequate for the 18-choice cap.

where each function fits

make_options builds ordinary Session records. session_values assigns the score of each opportunity. landscape computes all subsets, scores, violations, total time & energy. exact_schedule takes the best valid table row. greedy_schedule walks through valid one-session additions. package translates a selected row into topic minutes & chronological session records.

compare_schedule builds the same table for the quantum attempt. state_at maps two angles to a Statevector. objective maps those amplitudes to expected energy. minimize calls objective repeatedly. sample_counts returns measured strings & frequencies. only valid measured strings compete for best_sample. app.py stores the snapshot/result & dashboard.py renders them. the benchmark runner repeats compare_schedule across fixed cases & seeds.

the schedule energy includes higher-order rules such as any eligible review & rounded budget excess. this version applies its diagonal phase directly from the energy table. it does not expand these rules into the original nine-qubit pair-coefficient model. materializing all subsets costs exponential memory; larger real instances need a different cost representation & a scalable classical reference.

code appendix scheduler.py

actual source - src/optimizer/scheduler.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

```
001  """interacting session selection - overlaps, prerequisites & deadlines"""
002  from datetime import datetime,timezone
003  from importlib.metadata import version
004  from pathlib import Path
005  from time import perf_counter
006  import subprocess
008  import numpy as np
009  from pydantic import BaseModel,Field,model_validator
010  from qiskit import QuantumCircuit
011  from qiskit.quantum_info import Statevector
012  from scipy.optimize import minimize
014  from src.contracts import StudySnapshot
015  from src.optimizer.model import TOPICS
```

decision - define a shared record whose fields are checked by pydantic.

```
018  class Window(BaseModel):
019      start: int = Field(ge=0,le=10080)
020      end: int = Field(gt=0,le=10080)
022      @model_validator(mode="after")
```

decision - group this operation behind a reusable function.

```
023      def ordered(self):
024          if self.end<=self.start:
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
025              raise ValueError("window end must follow its start")
026          return self
```

decision - define a shared record whose fields are checked by pydantic.

```
029  class Session(BaseModel):
030      session_id: str
031      topic_id: str
032      kind: str
033      start: int = Field(ge=0)
034      minutes: int = Field(ge=15,le=45)
035      requires: list[str] = Field(default_factory=list)
```

```
    @model_validator(mode="after")
```

decision - group this operation behind a reusable function.

```
    def valid(self):
        if self.topic_id not in TOPICS or self.kind not in ("review","practice") or self.minutes%15 or self.start%15:
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
            raise ValueError("sessions need a topic, review/practice & 15-minute times")
        return self
```

decision - define a shared record whose fields are checked by pydantic.

```
class ScheduleSnapshot(BaseModel):
    schema_version: int = 3
    profile: StudySnapshot
    sessions: list[Session] = Field(min_length=1,max_length=18)
    windows: list[Window]
    deadlines: dict[str,int]
    @model_validator(mode="after")
```

decision - group this operation behind a reusable function.

```
    def valid(self):
        if self.schema_version!=3 or set(self.deadlines)!=set(TOPICS):
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
            raise ValueError("invalid scheduling version or topic deadlines")
        ids=[s.session_id for s in self.sessions]
        if len(ids)!=len(set(ids)):
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
            raise ValueError("session ids must be unique")
        for s in self.sessions:
            if any(r not in ids for r in s.requires):
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
                raise ValueError("unknown preparation session")
        return self
```

decision - construct candidate sessions, available windows & preparation alternatives from the saved profile.

```
def make_options(profile,start_hour=9,end_hour=13,exam_days=None,prepare=True,days=3,session_minutes=45):
    if end_hour<=start_hour:
```

decision - stop here with a readable reason; silently proceeding would hide an invalid input.

```
        raise ValueError("available time must end after it starts")
    exam_days=exam_days or {"T1":2,"T2":3,"T3":3}
    windows=[Window(start=d*1440+start_hour*60,end=d*1440+end_hour*60) for d in range(days)]
    sessions=[]
```

```
    for day in range(days):
        order=TOPICS[day%3:]+TOPICS[:day%3]
       for i,(kind,offset) in enumerate(zip(["review"]*3+["practice"]*3,[0,30,60,120,150,180])):
            topic=order[i%3]
            sessions.append(Session(session_id=f"d{day+1}-{topic}-{kind}",topic_id=topic,kind=kind,start=day*1440+start_hour*60+offset,minutes=session_minutes))
    if prepare:
        for s in sessions:
            if s.kind=="practice":
```

decision - store alternative eligible preparation session ids.

```
                s.requires=[r.session_id for r in sessions if r.topic_id==s.topic_id and r.kind=="review" and r.start+r.minutes+15<=s.start]
    return ScheduleSnapshot(profile=profile,sessions=sessions,windows=windows,deadlines={t:(exam_days[t]-1)*1440+end_hour*60 for t in TOPICS})
```

decision - calculate diagnostic gap, exam urgency & activity values.

```
def session_values(snapshot):
    profile=snapshot.profile
    values=[]
    for s in snapshot.sessions:
        gap=profile.topic_weights[s.topic_id]*(1-profile.request.proficiency[s.topic_id])
        days=max(0,(snapshot.deadlines[s.topic_id]-s.start)/1440)
        urgency=1+1/(1+days)
        values.append(gap*urgency*(1.4 if s.kind=="practice" else 1.0)*(s.minutes/45))
    return np.array(values)
```

decision - score every small candidate subset & mark all constraint violations.

```
def landscape(snapshot):
    snapshot=ScheduleSnapshot.model_validate(snapshot)
    sessions=snapshot.sessions
    n=len(sessions)
```

decision - enumerate integer subset indices for this small simulator model.

```
    indices=np.arange(1<<n,dtype=np.uint32)
```

decision - extract each candidate's on/off decision across every subset.

```
    bits=((indices[:,None]>>np.arange(n))&1).astype(np.int8)
    values=session_values(snapshot)
```

decision - sum selected session values using matrix multiplication.

```
    utility=bits@values
    violations=np.zeros(len(indices),dtype=np.int16)
    ids={s.session_id:i for i,s in enumerate(sessions)}
    conflicts=[]
```

```
    for i,s in enumerate(sessions):
        allowed=any(w.start<=s.start and s.start+s.minutes<=w.end for w in snapshot.windows)
        if not allowed or s.start+s.minutes>snapshot.deadlines[s.topic_id]:
```
decision - add a penalty whenever this invalid candidate is selected.
```
            violations+=bits[:,i]
        if s.requires:
            eligible=[ids[r] for r in s.requires if
sessions[ids[r]].start+sessions[ids[r]].minutes+15<=s.start]
```
decision - accept a preparation requirement when any eligible earlier review is selected.
```
            satisfied=bits[:,eligible].any(axis=1) if eligible else
np.zeros(len(bits),dtype=bool)
```
decision - add a penalty whenever this invalid candidate is selected.
```
            violations+=bits[:,i]*(~satisfied)
        for j in range(i):
            other=sessions[j]
            if s.start<other.start+other.minutes+15 and other.start<s.start+s.minutes+15:
```
decision - add a penalty whenever this invalid candidate is selected.
```
                violations+=bits[:,i]*bits[:,j]
                conflicts.append((j,i))
            if s.topic_id==other.topic_id and s.kind==other.kind:
```
decision - reduce repeated same-topic/activity value through pair interactions.
```
                utility-=.2*min(values[i],values[j])*bits[:,i]*bits[:,j]
    study=bits@np.array([s.minutes for s in sessions],dtype=np.int16)
    used_days=np.zeros(len(indices),dtype=np.int16)
    for day in sorted({s.start//1440 for s in sessions}):
```
decision - count days containing at least one selected session.
```
        used_days+=bits[:,[i for i,s in enumerate(sessions) if s.start//1440==day]].any(axis=1)
```
decision - charge breaks between selected sessions on a day, omitting that day's final break.
```
    breaks=15*(bits.sum(axis=1)-used_days)
    total=study+breaks
```
decision - round any positive budget excess upward, including non-multiple budgets.
```
    over=(np.maximum(0,total-snapshot.profile.request.budget_minutes)+14)//15
    violations+=over*over
    valid=violations==0
    penalty=1+float(values.sum())
```
decision - make valuable feasible schedules low energy & invalid schedules expensive.
```
    energies=-utility+penalty*violations
   return {"indices":indices,"bits":bits,"utility":utility,"valid":valid,"study":study,"breaks":
breaks,"total":total,"energies":energies,"values":values,"conflicts":conflicts,"penalty":penalty}
```

decision - rank candidate score, time cost & deterministic earlier-session ties.

```
133  def selection_key(index,data):
134      return (float(data["utility"][index]),-int(data["total"][index]),*data["bits"][index].tolist
())
```

decision - convert a bit selection into the schedule & time accounting shown by the app.

```
137  def package(snapshot,index,data,solver,elapsed=0):
138      selected=[s for i,s in enumerate(snapshot.sessions) if data["bits"][index,i]]
139      selected.sort(key=lambda s:(s.start,s.session_id))
140      allocs={t:sum(s.minutes for s in selected if s.topic_id==t) for t in TOPICS}
141      return {"allocations":allocs,"selected_sessions":[s.model_dump() for s in
selected],"used_minutes":int(data["study"][index]),"break_minutes":int(data["breaks"][index]),"unused
_minutes":snapshot.profile.request.budget_minutes-int(data["total"][index]),"utility":float(data["uti
lity"][index]),"feasible":bool(data["valid"][index]),"actual_solver":solver,"execution_mode":"live","
elapsed_ms":elapsed}
```

decision - find the highest-scoring feasible subset in the bounded reference instance.

```
144  def exact_schedule(snapshot,data=None):
145      start=perf_counter()
146      data=landscape(snapshot) if data is None else data
147      best=float(data["utility"][data["valid"]].max())
```

decision - collect feasible candidates whose score matches the optimum within tolerance.

```
148      tied=np.flatnonzero(data["valid"] & (np.abs(data["utility"]-best)<=1e-10))
149      index=max(tied,key=lambda i:(-int(data["total"][i]),*data["bits"][i].tolist()))
150      return package(snapshot,int(index),data,"exact",(perf_counter()-start)*1000)
```

decision - add the next compatible score improvement until no addition helps.

```
153  def greedy_schedule(snapshot,data=None):
154      start=perf_counter()
155      data=landscape(snapshot) if data is None else data
156      index=0
157      while True:
158          candidates=[index|(1<<i) for i in range(len(snapshot.sessions)) if not index&(1<<i)]
159          candidates=[i for i in candidates if data["valid"][i] and
data["utility"][i]>data["utility"][index]+1e-12]
160          if not candidates:
161              break
162          index=max(candidates,key=lambda i:selection_key(i,data))
163      return package(snapshot,index,data,"greedy",(perf_counter()-start)*1000)
```

decision - run one bounded Qiskit comparison & preserve its inputs and actual samples.

```
166  def compare_schedule(snapshot,seed=7,evaluations=30,shots=1024):
167      snapshot=ScheduleSnapshot.model_validate(snapshot)
168      exact=exact_schedule(snapshot)
```

```
    start=perf_counter()
    data=landscape(snapshot)
    n=len(snapshot.sessions)
    energies=data["energies"]
    uniform_state=Statevector.from_label("+"*n)
```

decision - choose an initial selection probability from budget & average session cost.

```
    occupancy=min(.35,max(.02,snapshot.profile.request.budget_minutes/(float(np.mean([s.minutes for s in snapshot.sessions]))+15)/n))
    preparation=QuantumCircuit(n)
```

decision - prepare a budget-aware product state with actual Qiskit rotation gates.

```
    preparation.ry(2*np.arcsin(np.sqrt(occupancy)),range(n))
```

decision - simulate the preparation circuit to obtain initial amplitudes.

```
    initial_state=Statevector.from_instruction(preparation)
    initial=np.random.default_rng(seed).uniform(0,np.pi,2)
```

decision - seed the measurement sampler for this final distribution.

```
    uniform_state.seed(seed)
```

decision - retain an unoptimized uniform-sampling reference with the same shot count.

```
    uniform_counts={key:int(val) for key,val in uniform_state.sample_counts(shots).items()}
```

decision - filter actual samples through the independently defined scheduling constraints.

```
    uniform_valid=[int(key,2) for key in uniform_counts if data["valid"][int(key,2)]]
    uniform_best=max(uniform_valid,key=lambda i:selection_key(i,data)) if uniform_valid else None
```

decision - filter actual samples through the independently defined scheduling constraints.

```
    uniform_rate=sum(count for key,count in uniform_counts.items() if data["valid"][int(key,2)])/shots
    trace=[]
```

decision - apply an energy-dependent phase followed by Qiskit X mixer rotations.

```
    def state_at(angles):
        beta,gamma=angles
```

decision - apply the scaled diagonal cost phase to each amplitude.

```
        phased=Statevector(initial_state.data*np.exp(-1j*gamma*energies/data["penalty"]))
        mixer=QuantumCircuit(n)
```

decision - apply the standard QAOA X mixer with Qiskit.

```
        mixer.rx(2*beta,range(n))
        return phased.evolve(mixer)
```

decision - give scipy a scalar expected energy for its current angle choice.

```
    def objective(angles):
        state=state_at(angles)
```

decision - return expected energy, the distribution's probability-weighted average.

```
        energy=float(np.dot(state.probabilities(),energies))
```

decision - record each objective call for the evaluation-budget audit.

```
        trace.append({"params":[float(a) for a in angles],"energy":energy})
        return energy
```

decision - cap this optimizer at 30 objective calls & suppress routine output.

```
    opt=minimize(objective,initial,method="COBYLA",options={"maxiter":evaluations,"disp":False})
    state=state_at(opt.x)
```

decision - seed the measurement sampler for this final distribution.

```
    state.seed(seed)
    counts={key:int(val) for key,val in state.sample_counts(shots).items()}
```

decision - filter actual samples through the independently defined scheduling constraints.

```
    feasible=[int(key,2) for key in counts if data["valid"][int(key,2)]]
    best=max(feasible,key=lambda i:selection_key(i,data)) if feasible else None
    sampled=None if best is None else package(snapshot,best,data,"qaoa")
```

decision - filter actual samples through the independently defined scheduling constraints.

```
    rate=sum(count for key,count in counts.items() if data["valid"][int(key,2)])/shots
    elapsed=(perf_counter()-start)*1000
    root=Path(__file__).resolve().parents[2]
    commit=subprocess.run(["git","rev-parse","HEAD"],cwd=root,capture_output=True,text=True,chec
k=True).stdout.strip()
    dirty=bool(subprocess.run(["git","status","--porcelain"],cwd=root,capture_output=True,text=T
rue,check=True).stdout.strip())
```

decision - keep utility null when no feasible sample exists.

```
   return {"schema_version":3,"created_utc":datetime.now(timezone.utc).isoformat(),"code_commit"
:commit,"working_tree_dirty":dirty,"profile":snapshot.model_dump(mode="json"),"execution_mode":"live",
"reference":"https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorit
hm","versions":{name:version(name) for name in ("qiskit","numpy","scipy")},"exact":exact,"greedy":gre
edy_schedule(snapshot),"uniform":{"raw_best_utility":None if uniform_best is None else
float(data["utility"][uniform_best]),"feasible_shot_rate":uniform_rate},"quantum":{"qubits":n,"p":1,"
seed":seed,"shots":shots,"objective_evaluations":len(trace),"initial_params":initial.tolist(),"initia
l_selection_probability":occupancy,"cost_scale":data["penalty"],"parameter_order":["beta","gamma"],"p
arams":opt.x.tolist(),"optimizer_success":bool(opt.success),"expected_energy":float(opt.fun),"best_sa
mple":sampled,"raw_best_utility":None if sampled is None else
sampled["utility"],"feasible_shot_rate":rate,"elapsed_ms":elapsed,"counts":counts,"trace":trace},"abs
olute_utility_gap":None if sampled is None else abs(exact["utility"]-sampled["utility"])}
```

code appendix
benchmark_scheduler.py

actual source - scripts/benchmark_scheduler.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

```
001  """fixed fictional scheduling cases, three seeds each"""
002  import argparse
003  import json
004  from pathlib import Path
005  import sys
007  ROOT=Path(__file__).resolve().parents[1]
008  if str(ROOT) not in sys.path:
009      sys.path.insert(0,str(ROOT))
011  from src.intelligence.pipeline import load_course
012  from src.intelligence.assess import score_diagnostic
013  from src.optimizer.sessions import make_study_snapshot
014  from src.optimizer.scheduler import make_options,compare_schedule
```

decision - connect the command line to the implementation without importing the ui.

```
017  def main():
018      parser=argparse.ArgumentParser(description="compare schedulers on fixed fictional cases")
```

decision - declare an input/output path option rather than hard-coding a student file.

```
019      parser.add_argument("--output",type=Path,default=ROOT/"runtime/scheduler_benchmark.json")
020      args=parser.parse_args()
021      course=load_course(ROOT/"data/demo/course.json")
022     profiles={"weak_integrals":{"T1":1.,"T2":0.,"T3":.5},"mixed_gaps":{"T1":.5,"T2":0.,"T3":0.},"
weak_derivatives":{"T1":0.,"T2":.5,"T3":1.}}
023      results=[]
024      examples=[]
025      for label,target in profiles.items():
026          answers={}
027          used=dict.fromkeys(target,0)
028          for q in course.questions:
029              if q.purpose!="diagnostic":
030                  continue
031              correct=used[q.topic_id]<int(2*target[q.topic_id])
```

```
            answers[q.question_id]=q.correct_choice if correct else next(c for c in q.choices if c!=q.correct_choice)
            used[q.topic_id]+=1
        scores=score_diagnostic(course,answers).proficiency
        for size,budget in ((12,240),(15,360),(18,600)):
            snapshot=make_options(make_study_snapshot(course,scores,budget))
            snapshot=snapshot.model_copy(update={"sessions":snapshot.sessions[:size]})
            examples.append({"case":f"{label}_{size}","answers":answers,"profile":snapshot.model_dump(mode="json")})
            for seed in (7,11,23):
                report=compare_schedule(snapshot,seed=seed)
                q=report["quantum"]
                results.append({"case":f"{label}_{size}","choices":size,"budget_minutes":budget,"seed":seed,"exact":report["exact"]["utility"],"greedy":report["greedy"]["utility"],"qaoa":q["raw_best_utility"],"uniform":report["uniform"]["raw_best_utility"],"feasible_rate":q["feasible_shot_rate"],"uniform_feasible_rate":report["uniform"]["feasible_shot_rate"],"gap":report["absolute_utility_gap"],"exact_ms":report["exact"]["elapsed_ms"],"qaoa_ms":q["elapsed_ms"],"evaluations":q["objective_evaluations"],"initial_selection_probability":q["initial_selection_probability"],"code_commit":report["code_commit"],"working_tree_dirty":report["working_tree_dirty"]})
```

decision - show the actual result or status in the terminal; this does not change the calculation.

```
                print(f"{label} {size} seed {seed} - exact {report['exact']['utility']:.3f}, greedy {report['greedy']['utility']:.3f}, qaoa {q['raw_best_utility']}, feasible {q['feasible_shot_rate']:.1%}",flush=True)
                if label=="mixed_gaps" and size==18 and seed==7:
                    full=args.output.with_name("scheduler_example.json")
                    full.parent.mkdir(parents=True,exist_ok=True)
                    full.write_text(json.dumps(report,indent=2,allow_nan=False)+"\n")
```

decision - create missing report folders without deleting anything.

```
    args.output.parent.mkdir(parents=True,exist_ok=True)
```

decision - save every configured result & case for reproducibility.

```
    args.output.write_text(json.dumps({"model_version":3,"cases":examples,"settings":{"initial_state":"budget-aware independent ry rotations","seeds":[7,11,23],"p":1,"max_evaluations":30,"shots":1024},"results":results},indent=2,allow_nan=False)+"\n")
if __name__=="__main__":
    main()
```

code appendix app.py

actual source - app.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

```
001  """study dashboard - schema 2, short diagnostic & break-aware sessions"""
002  import json
003  from pathlib import Path
004  from types import SimpleNamespace
006  import streamlit as st
008  from src.intelligence.pipeline import load_course
009  from src.intelligence.rank import select_questions
010  from src.optimizer.sessions import make_study_snapshot,solve_session
011  from src.optimizer.scheduler import make_options,exact_schedule,compare_schedule
012  from src.ui.study_inputs import clear_plan,open_diagnostic,diagnostic_dialog
013  from src.ui.dashboard import theme,duration,render_schedule,render_practice
015  ROOT=Path(__file__).resolve().parent
016  st.set_page_config(page_title="ExamAce · your study desk",page_icon="■",layout="wide")
017  theme()
018  try:
019      course=load_course(ROOT/"data/demo/course.json")
020  except (ValueError,OSError) as error:
021      st.error(f"course data could not be loaded - {error}")
022      st.stop()
024  if "diag_seen" not in st.session_state:
025      st.session_state.update(diag_seen=True,diag_open=True,diag_step=0,diag_answers={})
027  with st.sidebar:
028      st.markdown("## examace")
029      st.caption("CALCULUS · STUDY DESK")
030      st.divider()
031      st.markdown("#### Make time for it")
032      hours=st.number_input("Hours available",min_value=0,max_value=24,value=2,step=1,key="study_h
ours",on_change=clear_plan)
033      mins=st.selectbox("Extra minutes",options=[0] if hours==24 else
[0,15,30,45],key="study_mins",on_change=clear_plan)
034      budget=hours*60+mins
035      st.caption(f"{duration(budget)} total · breaks included")
```

```
036      st.caption("15-minute breaks between sessions")
037      with st.expander("Availability & exams"):
038          session_minutes=st.selectbox("Focus length",[15,30,45],index=2,format_func=lambda
m:f"{m} minutes",key="session_minutes",on_change=clear_plan)
039          days=st.selectbox("Planning days",[1,2,3],index=2,key="planning_days",on_change=clear_pl
an)
040          start_hour=st.selectbox("Available from",list(range(6,22)),index=3,format_func=lambda
h:f"{h:02d}:00",key="available_start",on_change=clear_plan)
041          end_hour=st.selectbox("Available until",list(range(7,25)),index=6,format_func=lambda
h:f"{h:02d}:00",key="available_end",on_change=clear_plan)
042          exam_days={}
043          for topic in course.topics:
044              exam_days[topic.topic_id]=st.selectbox(f"{topic.name} exam",[1,2,3],index=1 if
topic.topic_id=="T1" else 2,format_func=lambda d:f"Day {d} ·
{end_hour:02d}:00",key=f"exam_{topic.topic_id}",on_change=clear_plan)
045          prepare=st.checkbox("Review before practice",value=True,key="prepare",on_change=clear_pl
an)
046      st.divider()
047      if st.button("Retake diagnostic",key="retake",use_container_width=True):
048          open_diagnostic()
049          st.rerun()
050      if "proficiency" in st.session_state:
051          st.caption("Initial topic score from two questions")
052          for topic in course.topics:
053              st.write(f"{topic.name} · {st.session_state['proficiency'][topic.topic_id]:.0%}")
055  st.markdown('<div class="eyebrow">calculus / study session</div>',unsafe_allow_html=True)
056  st.title("Your exam-week plan")
057  st.write("Choose your availability. Fit study around your exams.")
058  if "proficiency" not in st.session_state:
059      st.info("Start with six quick questions so the plan reflects your starting point.")
060      if st.button("Take the diagnostic",key="start_diagnostic",type="primary"):
061          open_diagnostic()
062          st.rerun()
063  else:
064      if st.button("Build my plan",key="build_plan",type="primary"):
065          try:
066              profile=make_study_snapshot(course,st.session_state["proficiency"],budget)
067              snapshot=make_options(profile,start_hour,end_hour,exam_days,prepare,days,session_min
utes)
068              result=exact_schedule(snapshot)
```

```
069              items=select_questions(course,SimpleNamespace(allocations=result["allocations"],feas
ible=result["feasible"]),profile.request.proficiency)
070              st.session_state["study_plan"]={"snapshot":snapshot,"result":result,"items":items}
071              st.session_state.pop("experiment",None)
072          except ValueError as error:
073              st.error(str(error))
075  plan_tab,practice_tab,experiment_tab=st.tabs(["Study plan","Practice","Quantum lab"])
076  saved=st.session_state.get("study_plan")
077  with plan_tab:
078      if saved:
079          render_schedule(saved["result"],{t.topic_id:t.name for t in
course.topics},saved["snapshot"])
080          st.download_button("Save session profile",data=saved["snapshot"].model_dump_json(indent=
2),file_name="student_profile.json",mime="application/json",key="session_download")
081          with st.expander("How the allocation works"):
082              st.write("Topic scores & exam dates set the priorities. The planner picks a
combination that fits your availability. It keeps review ahead of practice & leaves space for
breaks.")
083      else:
084          st.caption("Your plan will appear here after the diagnostic & a session budget.")
085  with practice_tab:
086      if saved:
087          render_practice(course,saved["items"],saved["result"])
088      else:
089          st.info("Build a session to see practice for its allocated topics.")
090  with experiment_tab:
091      st.markdown("### A different solver")
092      st.write("Compare your plan with a local QAOA simulation as an experiment.")
093      if saved and st.button("Run a local comparison",key="run_quantum"):
094          with st.spinner("Simulating your session…"):
095              st.session_state["experiment"]=compare_schedule(saved["snapshot"])
096      report=st.session_state.get("experiment")
097      if report:
098          quantum=report["quantum"]
099          a,b,c=st.columns(3)
100          a.metric("Exact score",f"{report['exact']['utility']:.3f}")
101          b.metric("Greedy score",f"{report['greedy']['utility']:.3f}")
102          c.metric("QAOA score","no feasible sample" if quantum['raw_best_utility'] is None else
f"{quantum['raw_best_utility']:.3f}")
```

```
103          st.caption(f"Feasible samples · {quantum['feasible_shot_rate']:.1%}")

104          st.caption(f"{quantum['qubits']} qubits · {quantum['shots']:,} samples")

105          if quantum["best_sample"]:

106              for topic in course.topics:

107                  st.write(f"**{topic.name}** · {duration(quantum['best_sample']['allocations'][to
pic.topic_id])}")

108          else:

109              st.warning("The simulation did not find a plan that fits. Try another comparison.")

110          st.download_button("Save comparison",data=json.dumps(report,indent=2),file_name="session
_comparison.json",mime="application/json",key="comparison_download")

112      benchmark_path=ROOT/"evidence/risi/scheduler_benchmark.json"

113      if benchmark_path.exists():

114          with st.expander("Saved example runs"):

115              benchmark=json.loads(benchmark_path.read_text())

116              rows=[{"Case":r["case"].replace("_"," "
),"Options":r["choices"],"Seed":r["seed"],"Exact":round(r["exact"],3),"Greedy":round(r["greedy"],3),"
QAOA":None if r["qaoa"] is None else round(r["qaoa"],3),"Feasible
samples":f"{r['feasible_rate']:.1%}"} for r in benchmark["results"]]

117              st.dataframe(rows,hide_index=True,use_container_width=True)

119  if st.session_state.get("diag_open",False):

120      diagnostic_dialog(course)
```

code appendix dashboard.py

actual source - src/ui/dashboard.py. line numbers refer to this source snapshot. comments & docstrings are included exactly as written. the explanatory notes describe intent; they are not a claim that a human personally checked the code.

```
001  """quiet session dashboard, plan & practice views"""

002  import html

003  import streamlit as st
```

decision - group this operation behind a reusable function.

```
006  def theme():

007      st.markdown("""<style>

008      .stApp {background:#f6f5f0;}

009      .block-container {max-width:1120px;padding-top:2.2rem;padding-bottom:3rem;}

010      h1,h2,h3 {letter-spacing:-.035em;color:#26382e;}

011      h1 {font-family:Georgia,serif;font-weight:500!important;font-size:2.8rem!important;}

012      h2 {font-size:1.6rem!important;}

013      [data-testid="stMetric"] {background:white;border:1px solid
#e0e6df;border-radius:12px;padding:16px;}

014      [data-testid="stMetricValue"] {font-size:1.8rem;}

015      [data-baseweb="tab-list"] {gap:24px;background:transparent;border-bottom:1px solid #d9e1d9;}

016      [data-baseweb="tab"] {background:transparent;font-weight:600;padding:14px 0;}

017      [data-testid="stSidebar"] {border-right:1px solid #e0e6df;background:#eeefe8;}

018      .eyebrow {font-size:.72rem;text-transform:uppercase;letter-spacing:.15em;color:#71806c;margi
n-bottom:8px;}

019      .session-card {border-left:3px solid #698c70;background:white;padding:12px 17px;margin:8px
0;border-radius:0 10px 10px 0;}

020      .pause-card {border-left-color:#c4b68c;background:#efeee6;}

021      .session-time {font-size:.75rem;color:#73806e;margin-bottom:3px;}

022      .session-topic {font-size:1rem;font-weight:600;color:#26382e;}

023      .session-note {font-size:.8rem;color:#71806c;margin-top:3px;}

024      @media(max-width:700px){h1{font-size:2.1rem!important;}.block-container{padding-top:1rem;}[d
ata-baseweb="tab-list"]{gap:16px;}}

025      </style>""",unsafe_allow_html=True)
```

decision - group this operation behind a reusable function.

```
028  def duration(minutes):

029      hours,mins=divmod(minutes,60)

030      return f"{hours}h {mins:02d}m" if hours else f"{mins}m"
```

decision - group this operation behind a reusable function.

```
033  def clock(minutes):
034      return f"{minutes//60}:{minutes%60:02d}"
```

decision - group this operation behind a reusable function.

```
037  def render_plan(plan,names):
038      a,b,c=st.columns(3)
039      a.metric("Focused study",duration(plan["used_minutes"]))
040      b.metric("Breaks",duration(plan["break_minutes"]))
041      c.metric("Time left",duration(plan["unused_minutes"]))
042      st.markdown("### Your focus areas")
043      for topic,minutes in sorted(plan["allocations"].items(),key=lambda pair:(-pair[1],pair[0])):
044          if minutes:
045              st.write(f"**{names[topic]}** · {duration(minutes)}")
046              st.progress(minutes/max(plan["used_minutes"],1))
047      if not plan["used_minutes"]:
048          st.info("No study blocks allocated. Try a longer budget or retake the diagnostic if
these scores do not reflect where you need practice.")
049          return
050      st.markdown("### A session you can follow")
051      st.caption("times are elapsed from your start · a 15-minute break after each 45 minutes of
work")
052      grouped=[]
053      for event in plan["timeline"]:
054          if grouped and event["kind"]=="study" and grouped[-1]["kind"]=="study" and
event["topic_id"]==grouped[-1]["topic_id"]:
055              grouped[-1]["duration_minutes"]+=15
056          else:
057              grouped.append(event.copy())
058      for event in grouped:
059          start=event["start_minutes"]
060          end=start+event["duration_minutes"]
061          pause=event["kind"]=="break"
062          title="Take a break" if pause else names[event["topic_id"]]
063          note="step away, stretch, get some water" if pause else "review the topic, then work
through its practice questions"
064          st.markdown(f'<div class="session-card {"pause-card" if pause else ""}"><div
class="session-time">{clock(start)} – {clock(end)}</div><div
class="session-topic">{html.escape(title)}</div><div
class="session-note">{note}</div></div>',unsafe_allow_html=True)
```

decision - group this operation behind a reusable function.

```
def render_practice(course,items,plan):
    questions={q.question_id:q for q in course.questions}
    sources={s.source_id:(s.label or s.filename or s.source_id) for s in course.sources}
    names={t.topic_id:t.name for t in course.topics}
    st.markdown("### PRACTICE")
    st.caption("practice with a reviewed question bank.")
    if not items:
        st.info("Build a plan with allocated study time to see targeted practice.")
    for item in items:
        q=questions[item.question_id]
        with st.container(border=True):
            st.caption(f"{names[q.topic_id].upper()} · LEVEL {q.difficulty}")
            st.markdown(f"#### {q.text}")
            st.caption(f"{sources[q.source_id]} · page {q.page}")
            with st.expander("Why this question?"):
                st.write(item.reason)
            if q.answer is None:
                st.caption("answer unavailable")
            else:
                with st.expander("Check your answer"):
                    st.write(q.answer)
    for topic,minutes in plan["allocations"].items():
        count=sum(questions[i.question_id].topic_id==topic for i in items)
        if minutes and count<2:
            st.warning(f"{names[topic]} - only {count} unique practice questions available")
```

decision - display selected sessions in day/time order & show excluded-choice reasons.

```
def render_schedule(plan,names,snapshot):
    a,b,c=st.columns(3)
    a.metric("Focused study",duration(plan["used_minutes"]))
    b.metric("Breaks",duration(plan["break_minutes"]))
    c.metric("Time left",duration(plan["unused_minutes"]))
    if not plan["selected_sessions"]:
        st.info("No sessions fit these settings. Try more availability or a later exam day.")
        return
    selected=plan["selected_sessions"]
    current_day=None
```

```
    for i,s in enumerate(selected):
        day=s["start"]//1440
        if day!=current_day:
            st.markdown(f"### Day {day+1}")
            current_day=day
        begin=s["start"]%1440
        end=begin+s["minutes"]
        title=f"{names[s['topic_id']]} · {s['kind'].title()}"
        note="work through the topic's exercises" if s["kind"]=="practice" else "review your notes & key examples"
        st.markdown(f'<div class="session-card"><div class="session-time">{clock(begin)} – {clock(end)}</div><div class="session-topic">{html.escape(title)}</div><div class="session-note">{note}</div></div>',unsafe_allow_html=True)
        if i+1<len(selected) and selected[i+1]["start"]//1440==day:
            st.markdown(f'<div class="session-card pause-card"><div class="session-time">{clock(end)} – {clock(end+15)}</div><div class="session-topic">Take a break</div></div>',unsafe_allow_html=True)
    with st.expander("Other available sessions"):
        selected_ids={s["session_id"] for s in selected}
        rows=[]
        for s in snapshot.sessions:
            if s.session_id in selected_ids:
                continue
            if s.start+s.minutes>snapshot.deadlines[s.topic_id]:
                reason="after the exam deadline"
            elif not any(w.start<=s.start and s.start+s.minutes<=w.end for w in snapshot.windows):
                reason="outside your availability"
            elif s.requires and not any(r in selected_ids for r in s.requires):
                reason="preparation session not selected"
            elif any(s.start<o["start"]+o["minutes"]+15 and o["start"]<s.start+s.minutes+15 for o in selected):
                reason="overlaps a selected session or break"
            else:
                reason="another combination gives a higher total score"
            rows.append({"Day":s.start//1440+1,"Time":clock(s.start%1440),"Topic":names[s.topic_id],"Activity":s.kind,"Reason":reason})
        st.dataframe(rows,hide_index=True,use_container_width=True)
```